

Assignment 3: A Neural n -gram Language Model

1. Goal

Your group replaces the count tables of Assignment 2 with a small neural network. The model reads the same BPE tokens. It sees the previous $n - 1$ tokens and predicts the next one. You train it in PyTorch, measure it with the same perplexity function, and compare it against the count-based model.

This model is the baseline for Assignment 4. It has no attention and a fixed context window. Everything it can do comes from the embeddings and one hidden layer.

2. Rules

1. You may use PyTorch. Use `nn.Embedding`, `nn.Linear`, `nn.ReLU`, `nn.Dropout`, `F.softmax` and `F.cross_entropy`.
2. Do not use `nn.Transformer`, `nn.MultiheadAttention` or a pretrained model.
3. Use your own BPE tokenizer from Assignment 1 and your own `perplexity` and `generate` functions from Assignment 2. Do not rewrite them.
4. Use the validation set to choose hyperparameters. Use the test set once, at the end.
5. Make the run deterministic. Call `torch.manual_seed` and report the seed.
6. You may use an AI assistant. If you do, write one paragraph in the README that explains how it played a role in your implementation.

3. Data

3.1 Shakespeare

Use the same corpus and the same three splits as in Assignment 2. Encode the splits with your BPE tokenizer, with `<bos>` and `<eos>`, exactly as before.

3.2 A second domain: Wall Street Journal

You also need a test set from a different domain. Use the Penn Treebank test file. It contains 1989 Wall Street Journal material, without the linguistic annotation.

```
url = "https://raw.githubusercontent.com/wojzaremba/lstm/master/data/ptb.test.txt"
# download to data/ptb.test.txt, then check it:
import hashlib, pathlib
p = pathlib.Path("data/ptb.test.txt")
print(len(p.read_text().splitlines())) # expect 3761
print(hashlib.md5(p.read_bytes()).hexdigest()) # expect 8b80168b89c18661a38ef683c0dc3721
```

Run this check in the notebook. If the line count or the hash is different, the download failed. A common failure is an HTML error page saved as text, so print the first two lines too.

Two properties of this file need attention:

- The file already contains the literal string `<unk>` for rare words. Replace every literal `<unk>` with the word `unknown` before you encode. Otherwise it collides with your own `<unk>` token.
- The text is lowercase, has no punctuation, and writes every number as `N`. Apply your `clean` normalization to it anyway, so both corpora go through the same path.

Never train on this file. You only measure on it. Expect a much higher perplexity than on Shakespeare. That is the result, not a bug.

4. Where the code runs

This assignment runs on a CPU. A GPU is not needed. Write the device once and use it everywhere:

```
device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
# on an Apple laptop you may also try:
# device = "mps" if torch.backends.mps.is_available() else "cpu"

model = model.to(device)
xb, yb = xb.to(device), yb.to(device)  # every batch, every step
```

The model and the batch must be on the same device, or PyTorch raises an error about tensors on different devices. With this one line, the same notebook runs on a laptop and on a GPU machine. If you have a GPU, you may raise the model size and the number of steps, but the CPU configuration must still be the one in the notebook that we run.

Time limit. One training run must finish in under 5 minutes on your CPU. Report the time of one run. If a run is slower, lower `max_steps` or the hidden size.

5. Task 1: Batches (15 points)

The model sees a fixed window. One training example is $n - 1$ input tokens and one target token.

1. Turn each split into one long list of token ids. Keep the `<bos>` and `<eos>` tokens from Assignment 2.
2. Write `get_batch(split, batch_size)`. It returns `x` of shape `(batch_size, n-1)` and `y` of shape `(batch_size,)`, both `torch.long` and both on the device.
3. Print one batch with small numbers, and decode one row by hand, to show that the target is the token that follows the context.

6. Task 2: The model (25 points)

Write a class `NeuralNGramLM`. It takes n , the vocabulary size V , the embedding size and the hidden size.

1. `nn.Embedding(V, n_embd)` maps each of the $n - 1$ context ids to a vector.
2. Flatten the $n - 1$ vectors into one vector of length $(n - 1) \times n_embd$.
3. `nn.Linear` to the hidden size, then `nn.ReLU`.
4. `nn.Linear` from the hidden size to V . These are the logits.
5. `forward` takes `idx` and `targets`, and returns the logits and the loss. Compute the loss with `F.cross_entropy`. Return `None` for the loss when there are no targets.

The two methods that keep the old code working. Add `log_prob(token, context)` and `next_token_log_probs(context)`, with the same signatures as in Assignment 2. Then your `perplexity` and `generate` functions run on this model without a change. Put the model in `model.eval()` and wrap these methods in `torch.no_grad()`.

A loop over single tokens is slow. You may add a batched perplexity function for speed. If you do, show that it gives the same value as the Assignment 2 function to three decimals.

Sanity check. Before training, the model is close to uniform, so the loss should be near $\ln V$. Print it. A much lower value at step zero means the targets are leaking into the input.

7. Task 3: Training and evaluation (20 points)

1. Write the training loop. The order is: get a batch, forward, clear the gradients, backward, step. Use `torch.optim.AdamW`.
2. Write `estimate_loss()` with the `@torch.no_grad()` decorator. It averages the loss over several batches for the training split and the validation split. Call `model.eval()` before and `model.train()` after.
3. Print both losses every few hundred steps.
4. Plot the training loss and the validation loss against the step number, on one plot. Say in one sentence where the validation loss stops falling.

Start from this configuration: $n = 3$, $k = 1000$, $n_embd=64$, $n_hidden=256$, $batch_size=64$, $lr=1e-3$, $max_steps=2000$.

8. Task 4: Experiments (30 points)

Report the perplexity per token and the perplexity per character, as defined in Assignment 2. Use the perplexity per character whenever you compare across k .

Experiment 1, vary k . Train the same model with $k \in \{250, 1000, 4000\}$. Keep everything else fixed. Report both perplexities on the validation set and on the test set. Say in two sentences how k changes the result, and why the two perplexity variants disagree.

Experiment 2, counts against the network. Take your count-based n -gram model from Assignment 2 at the same n and the same k . Put the two models in one table: both perplexities on the Shakespeare test set, the number of parameters or counts stored, and the training time. Say in two sentences which model wins and what it costs.

Experiment 3, two hyperparameters. Choose two from this list: learning rate, batch size, embedding size, hidden size, number of steps. Try three values of each, one at a time, while the other stays at the starting value. That is six runs. Report the validation perplexity of each, and plot one line per hyperparameter. Say which of the two matters more.

9. Task 5: Generation and the second domain (10 points)

1. Take your best model from Task 4. Generate ten samples with `mode="sample"` and three with `mode="argmax"`, using the `generate` function from Assignment 2 and the same three prompts. Show them in the notebook.
2. Measure the count-based model from Assignment 2 and your best neural model on the Wall Street Journal test file. Put the four numbers, both perplexity variants for both models, in one table next to the Shakespeare numbers.
3. Say in two sentences why both models get worse, and which one gets worse faster.

10. Required interface

```
from bpe import BPETokenizer
from evaluation import perplexity, generate      # unchanged from Assignment 2
from neural_ngram import NeuralNGramLM

model = NeuralNGramLM(n=3, vocab_size=V, n_embd=64, n_hidden=256).to(device)
train(model, train_ids, max_steps=2000, lr=1e-3, batch_size=64)

model.log_prob(token_id, context_ids)
model.next_token_log_probs(context_ids)         # numpy array, length V

pp, pp_char = perplexity(model, test_ids)
text = generate(model, tok, "shall i compare thee", mode="sample", max_tokens=50, seed=0)
```

11. What you hand in

The deadline for all submissions is 30 September 2026 at 17:00. This applies to Assignment 1 to Assignment 4.

Make one ZIP file with the name `group_NN_a3.zip`. Upload it to the folder of your group on StudIP. One member uploads for the whole group. The ZIP contains:

- `assignment3.ipynb`, with all output cells kept. It must run from top to bottom after *Restart and Run All*, on a CPU.
- Your `.py` files, including the ones you reuse from Assignments 1 and 2, and `requirements.txt`.
- `README.md`, with the contribution table described below. Keep the ZIP under 10 MB.

The contribution statement. The `README.md` must start with a table in this form:

Name	Student number	Tasks	Share
Student 1	123456	Task 2, Task 3	40 %
Student 2	234567	Task 4, plots	35 %
Student 3	345678	Task 1, Task 5, README	25 %

The shares must sum to 100. All members must agree to the table. Add one sentence about the work of each member in your own words. We grade the submission as one piece of work, but we use this table in the review session, and we can adjust a single grade if the table and the review do not agree. A submission without this table loses 10 points.

The `README.md` also gives the total run time, the machine and the device that you used, the random seed, and one paragraph on how AI played a role in your implementation, if you used it. Do not hand in the corpora. If the full run takes more than 30 minutes, save the trained models and read them back, and add a flag `FORCE_RETRAIN = False` at the top of the notebook.

12. Grades

Item	Points
Task 1. Batches of the right shape, with a decoded example	15
Task 2. Correct model, the two interface methods, the $\ln V$ check	25
Task 3. Training loop, evaluation loop, loss plot	20
Task 4. The three experiments, with tables and plots	30
Task 5. Generation and the Wall Street Journal comparison	10
Total	100

We remove points for these problems: the notebook does not run from top to bottom, -15 ; it does not run on a CPU, -10 ; the contribution table is absent, -10 ; the old evaluation functions were rewritten instead of reused, -10 ; the test set is used to choose hyperparameters, -10 ; two runs give different numbers, -5 ; a plot has no axis label or no legend, -2 each.

13. The next assignment

Assignment 4 is the last one. You build a small GPT. It reads the same BPE tokens and it keeps the same interface, so your `perplexity` and `generate` functions work on it again. The difference is that the context stops being a fixed window of $n - 1$ tokens. Self-attention lets every position read every earlier position, and the model learns which ones matter. The neural n -gram you just built is the baseline it has to beat. Keep your code and your result tables. You will need both.